5G00GC11-3001 AI and Machine Learning

Spring 2026, 5 cr

AUTHOR
Teemu J. Heinimäki

AI & ML, Week 13, Part 2

Random forests

- In so-called ensemble methods, multiple models decide on the output collectively.
- Random forests are a family of ensemble methods that use decision trees (constructed applying some randomization) as their base learners.
  - A decision can be simply, e.g., an average value or the result of a majority vote.
  - It is good to have diverse trees in the forest.
    - Low correlation between trees is beneficial for the ensemble's performance – helps in reducing overfitting.
    - Getting diverse trees can be achieved by using different subsets of the training data for different trees or limiting the branching options.

Image produced using MS Copilot (Feb 9, 2025)

Going through the route-finding task

- Let us go through the last week's route-finding task.
- (Is there any other task so far we should check together?)

Example on a regression tree

- In regression trees (regression variants of decision trees), the feature values are used to split the data into subsets (partitioning the data recursively), and some simple model – for example, simply mean – is fitted to each of the resulting leaf subsets.

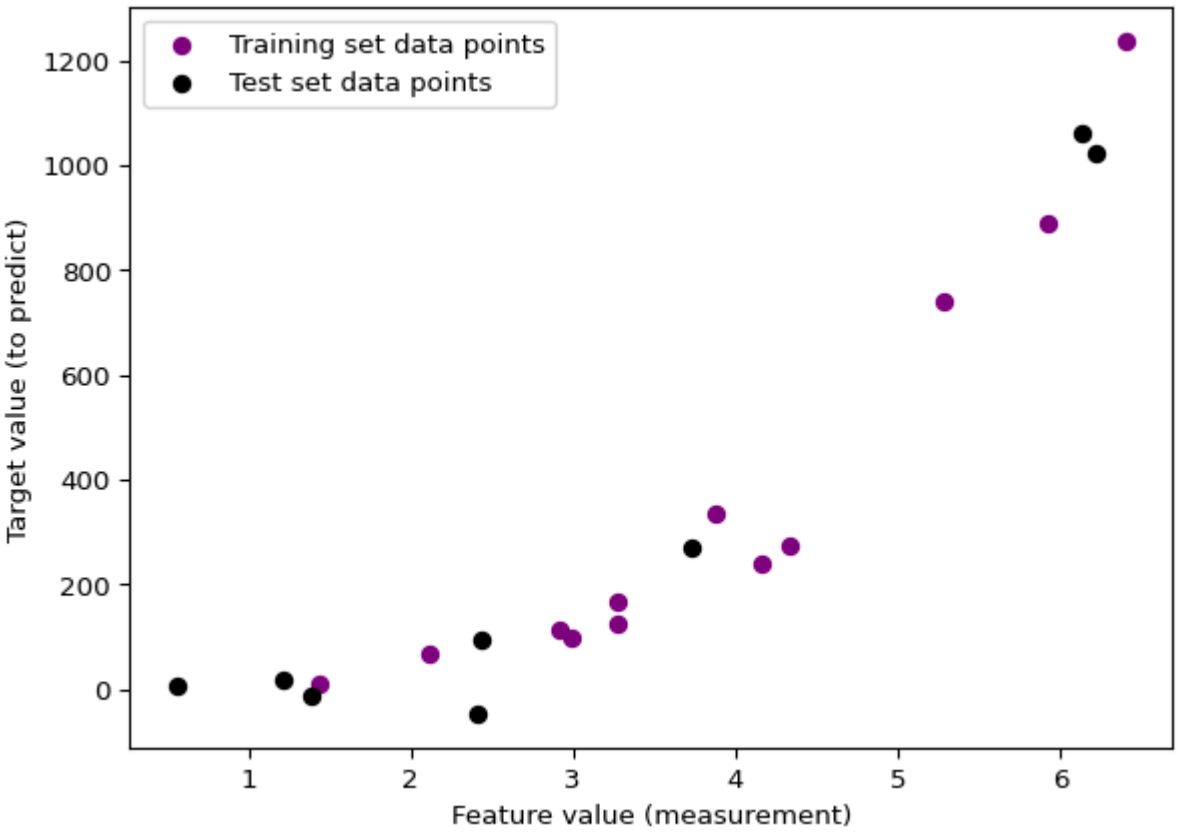
```
import numpy as np
from sklearn.tree import DecisionTreeRegressor, export_text, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

# Creating a synthetic dataset for regression
np.random.seed(747)
X = np.sort(7 * np.random.rand(20, 1), axis=0)
mu, sigma = 0, 50
```

```
# ravel: returns a flattened array
y = (5* X**3 - 3* X**2 + 2* X + 1).ravel() + np.random.normal(mu, sigma, X.shape[0])

# Splitting the dataset into training and testing sets
# (60% training, 40% testing)
# giving a random_state value tries to ensure reproducibility:
# same split each time the code is run with the same random_state value.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=
```

```
plt.scatter(X_train, y_train, color='purple', label='Training set data points')
plt.scatter(X_test, y_test, color='black', label='Test set data points')
plt.xlabel("Feature value (measurement)")
plt.ylabel("Target value (to predict)")
plt.legend()
plt.show()
```



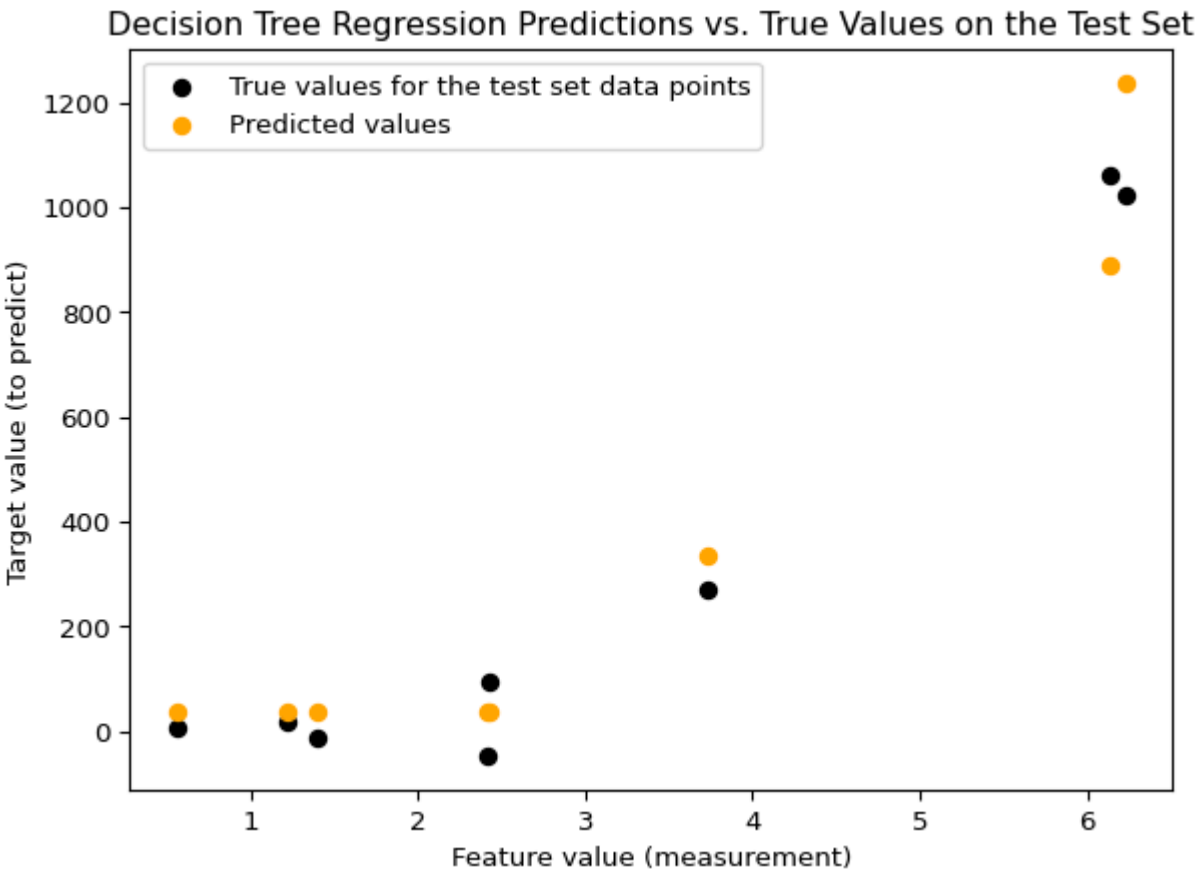

```
# Creating and training the Decision Tree Regressor
dt = DecisionTreeRegressor(max_depth=3, random_state=5)
dt.fit(X_train, y_train)

# Testing the model by making predictions on the test set
y_pred = dt.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"MSE: {mse:.3f}")

# Visualizing the predictions
plt.figure(figsize=(7, 5))
plt.scatter(X_test, y_test, color='black',
            label='True values for the test set data points')
```

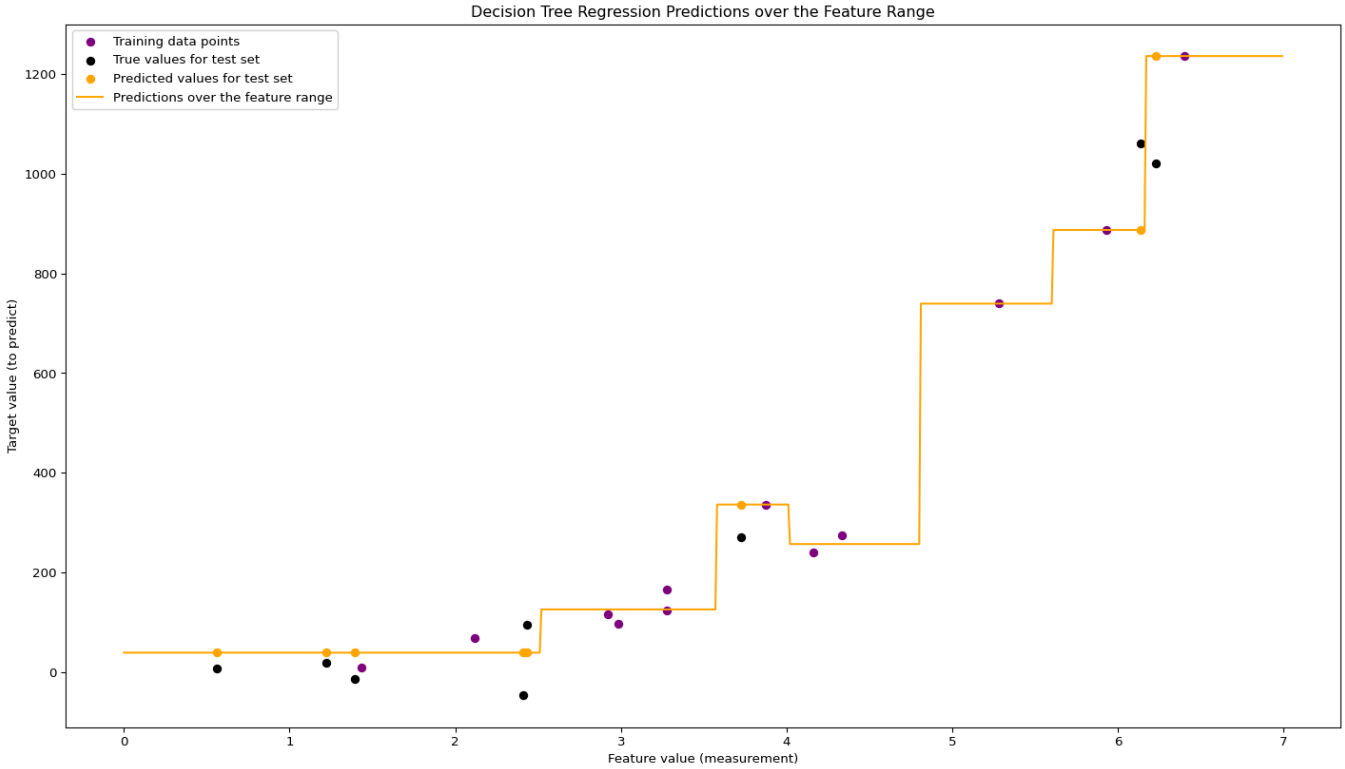
```
plt.scatter(X_test, y_pred, color='orange',
            label='Predicted values')
plt.xlabel("Feature value (measurement)")
plt.ylabel("Target value (to predict)")
plt.title("Decision Tree Regression Predictions vs. True Values on the Test Set")
plt.legend()
plt.show()
```

```
MSE: 11911.938
```



- Let us also visualize the predictions over the whole feature range, not just on the test set data points.

```
# Regression line
# np.newaxis is used to increase the dimension of the existing array by one more dimen
X_all = np.arange(0.0, 7.0, 0.01)[:, np.newaxis]
y_all_preds = dt.predict(X_all)
plt.figure(figsize=(18, 10))
plt.scatter(X_train, y_train, color='purple', label='Training data points')
plt.scatter(X_test, y_test, color='black', label='True values for test set')
plt.scatter(X_test, y_pred, color='orange', label='Predicted values for test set')
plt.plot(X_all, y_all_preds, color='orange', label='Predictions over the feature range
plt.xlabel("Feature value (measurement)")
plt.ylabel("Target value (to predict)")
plt.title("Decision Tree Regression Predictions over the Feature Range")
plt.legend()
plt.show()
```

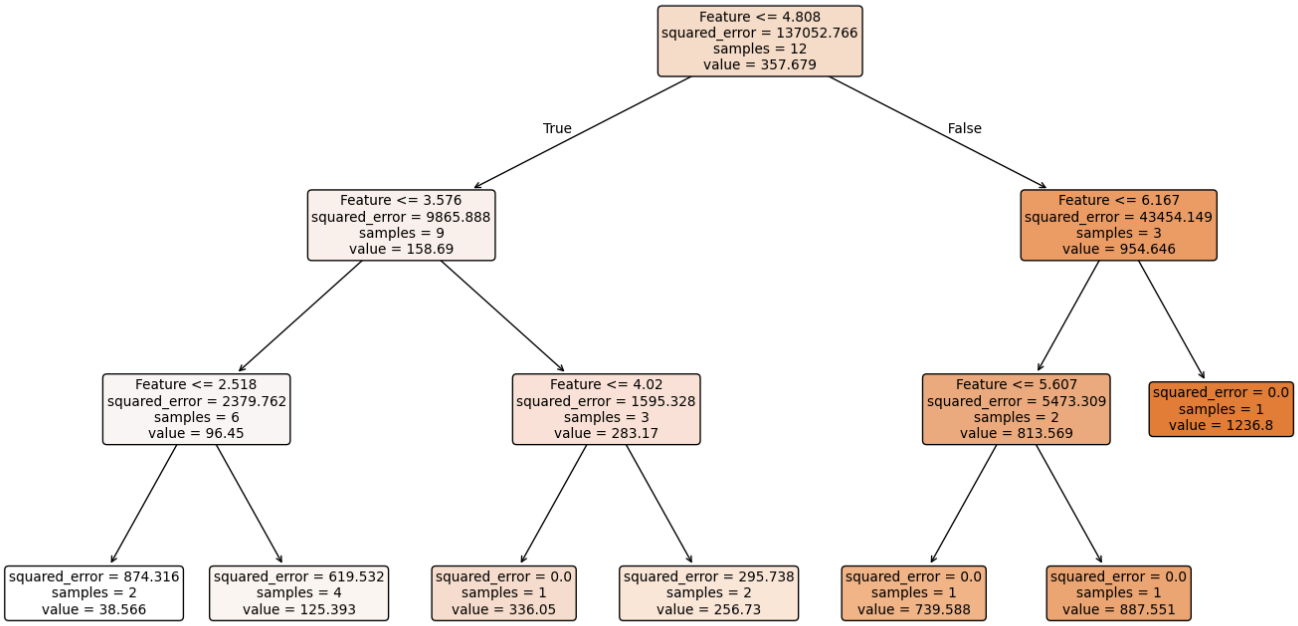



```
# Printing the decision tree rules/structure as text
tree_rules = export_text(dt, feature_names=['Feature'])
print(tree_rules)
```

```
|--- Feature <= 4.81
|   |--- Feature <= 3.58
|   |   |--- Feature <= 2.52
|   |   |   |--- value: [38.57]
|   |   |--- Feature >  2.52
|   |   |   |--- value: [125.39]
|   |--- Feature >  3.58
|   |   |--- Feature <= 4.02
|   |   |   |--- value: [336.05]
|   |   |--- Feature >  4.02
|   |   |   |--- value: [256.73]
|--- Feature >  4.81
|   |--- Feature <= 6.17
|   |   |--- Feature <= 5.61
|   |   |   |--- value: [739.59]
|   |   |--- Feature >  5.61
|   |   |   |--- value: [887.55]
|   |--- Feature >  6.17
|   |   |--- value: [1236.80]
```

```
# Visualizing the decision tree structure graphically
plt.figure(figsize=(18, 10))
plot_tree(dt, filled=True, feature_names=['Feature'],
          rounded=True)
plt.title("Decision Tree Structure")
plt.show()
```

Decision Tree Structure



Exercise

- Familiarize yourself with scikit-learn (`pip install scikit-learn`, if you haven't already). Notice importing with `import sklearn` (not `import scikit-learn`).
- Experiment with the classes `DecisionTreeRegressor` (https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html) and `RandomForestRegressor` (https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html):
  - Use data from the files `regression_exercise_training_data.csv` and `regression_exercise_test_data.csv` available in Moodle, and fit regression trees and random forests to the data using different parameters. Test at least different
    - `max_depth` values for the decision tree (e.g., 1, 3, 5, None) and
    - `n_estimators` values for the random forest (e.g., 1, 20, 100).
  - Compare the performances of the models on the test set (e.g., using mean squared error values).
  - If you wish, you can also visualize the predictions of the models and the decision tree structures.
- Are there other factors that affect the prediction quality and usefulness of the models besides the function parameters? (If, what?)

Thank you!